

LLM-Based Task Offloading and Resource Allocation in Satellite Edge Computing Networks

Minghao Sun¹, Jinbo Hou¹, Kehai Qiu¹,
 Kezhi Wang², *Senior Member, IEEE*,
 Xiaoli Chu¹, *Senior Member, IEEE*, and Zitian Zhang¹

Abstract—Satellite Mobile Edge Computing (MEC) networks offer a promising solution for delivering global services to terrestrial Internet of Things (IoT) terminals in 5G and beyond. However, satellite MEC systems face challenges such as underutilization of resources and task congestion, leading to resource waste and increased latency. In this paper, we investigate the joint resource allocation and task offloading problem in multi-satellite MEC networks, aiming to minimize the average latency of IoT terminals. To solve the joint optimization problem involving IoT terminals' task offloading decisions, uplink transmission power and sub-channel allocation, and satellite computation resource allocation, we propose an iterative optimization algorithm that uses the Lagrange multipliers method to optimize the satellite computation resource allocation and a Large Language Model (LLM) based optimizer to optimize the other variables in each iteration. Prompts and templated parameters are designed to enhance the LLM's inference accuracy and generalization capability across scenarios with varying numbers of satellites and IoT terminals. Simulation results show that our proposed LLM-based algorithm outperforms benchmark algorithms in convergence speed and average latency of IoT terminals.

Index Terms—Satellite mobile edge computing, task offloading, resource allocation, large language model, Internet of Things.

I. INTRODUCTION

Internet of Things (IoT) terminals have driven numerous intelligent applications [1]. However, terrestrial communication networks fail to provide reliable communication services for IoT terminals in remote areas, such as disaster zones, oceans, and deserts. Low-Earth-Orbit (LEO) satellite Mobile Edge Computing (MEC) networks can help provide global service coverage for IoT terminals [2], [3]. Nonetheless, achieving efficient resource allocation and task offloading while meeting low-latency requirements remains challenging due to limited communication and computation resources at both satellites and terminals [4].

Received 8 April 2025; revised 31 July 2025 and 13 September 2025; accepted 15 September 2025. Date of publication 19 September 2025; date of current version 6 March 2026. This work was supported in part by the Horizon Europe Research and Innovation Program under Grant 101086219 and Grant 101086228, in part by U.K. Engineering and Physical Sciences Research Council under Grant EP/X038971/1 and Grant EP/Y028031/1, in part by Royal Society International Exchanges Award under Grant IEC/NSFC/242607, in part by the Innovate U.K. COMET Project under Grant 10099265, and in part by the Royal Society Industry Fellowship under Grant IF R2 23200104. The review of this article was coordinated by Prof. Jiawen Kang. (*Corresponding authors: Xiaoli Chu; Zitian Zhang.*)

Minghao Sun, Jinbo Hou, and Xiaoli Chu are with the School of Electrical and Electronic Engineering, The University of Sheffield, S10 2TN Sheffield, U.K. (e-mail: msun39@sheffield.ac.uk; jhou9@sheffield.ac.uk; x.chu@sheffield.ac.uk).

Kehai Qiu is with the Department of Computer Science and Technology, University of Cambridge, CB2 1TN Cambridge, U.K., and also with the Brunel University of London, UB8 3PH Uxbridge, U.K. (e-mail: kq218@cam.ac.uk).

Kezhi Wang is with the Department of Computer Science, Brunel University of London, UB8 3PH Uxbridge, U.K. (e-mail: kezhi.wang@brunel.ac.uk).

Zitian Zhang is with the School of Information and Electronic Engineering, Zhejiang Gongshang University, Hangzhou 314423, China (e-mail: zitian.zhang@mail.zjgsu.edu.cn).

Digital Object Identifier 10.1109/TVT.2025.3612207

Some research has been conducted in this area. The authors in [5] minimized latency and energy consumption in a satellite MEC network by Breadth-First Search and greedy algorithms. The latency was minimized by using the Genetic Algorithm (GA) and Lagrange multiplier method in [6] and by employing game theory and many-to-one matching theory in [7]. The authors in [8] solved a weighted-sum latency minimization problem for satellite-assisted vehicle-to-vehicle networks by Reinforcement Learning. Under limited bandwidth, effective power and spectrum allocation schemes are necessary to overcome co-channel interference. However, the existing works [5], [6], [7], [8] did not consider the impact of transmission power and spectrum allocation on the data transmission rate or latency for offloading tasks from terrestrial terminals to satellites. Moreover, existing algorithms suffer from issues such as limited applicability, poor generalization, and slow convergence. Large Language Models (LLMs) have emerged as a promising approach to solve these issues with their contextual learning and inference abilities, which have demonstrated outstanding optimization capability for wireless networks [9], [10].

In this paper, we aim to minimize the average latency of IoT terminals in a multi-satellite MEC network by optimizing the satellites' computation resource allocation and the IoT terminals' task offloading decisions, uplink sub-channel allocation, and transmission power allocation. Given that the formulated problem is non-convex and challenging to solve directly, we decompose it into two sub-problems: the computation resource allocation problem and the joint task offloading, power allocation, and sub-channel allocation problem. The former is proven to be convex and can be solved using the Lagrange multiplier method to obtain a closed-form result. For the latter, due to its complexity, traditional optimization algorithms often suffer from prolonged computing time [11], while existing AI algorithms typically require substantial time for model training or fine-tuning [12]. By harnessing the LLM's inference and generalization capabilities while avoiding the costs of dedicated model training, we propose an LLM-based optimizer that utilizes structured templates, pre-designed prompts, and an example pool to solve the second subproblem. An alternating optimization algorithm is devised based on the solutions to both sub-problems to solve the original problem. Simulation results are provided to evaluate the proposed algorithm for varying numbers of satellites and IoT terminals.

II. SYSTEM MODEL AND PROBLEM FORMULATION

We consider a low-density satellite-MEC scenario in a remote area. The network contains M LEO satellites, denoted by $\mathcal{S} = \{S_1, S_2, \dots, S_M\}$, which serve N IoT terminals sparsely distributed on the ground at fixed locations via I orthogonal sub-channels, as shown in Fig. 1. Each satellite has a computational capacity F_{SAT} (in CPU cycle/s). The set of IoT terminals is represented by $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$. The n -th IoT terminal T_n has a task ψ_n with data size ε_n (in bits) and required computational density ρ (in CPU cycle/bit). Each terminal has a local computational capacity of F_{IoT} (in CPU cycle/s). An IoT terminal can either process its task locally or offload it to one of the satellites. The set of sub-channels is denoted by $\mathcal{A} = \{A_1, A_2, \dots, A_I\}$. Each sub-channel has a bandwidth of B and can be employed by multiple IoT terminals simultaneously. One IoT terminal can use multiple sub-channels to send the task data to one satellite and can allocate various power across the sub-channels, with the total transmission power constrained by P_{IoT} . The scheduling

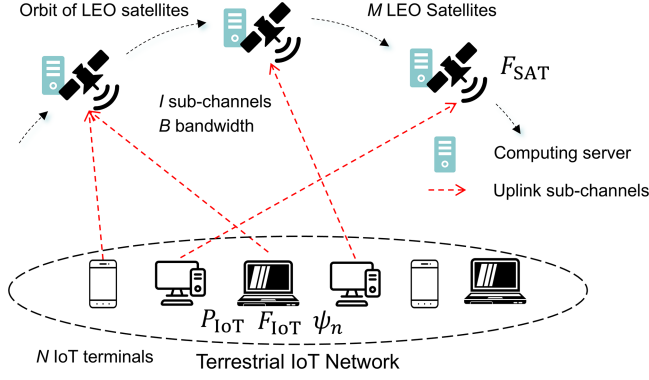


Fig. 1. Satellite Mobile Edge Computing Network.

period Δ is assumed to be sufficiently short such that the satellite-terminal geometry and channel fading coefficients can be regarded as quasi-static [11]. At the beginning of each scheduling period, the binary task offloading indicators $\alpha_{m,n}$ and sub-channel allocation indicators $\beta_{n,i}$ are updated by the system. Specifically, $\alpha_{m,n} = 1$ if task ψ_n is offloaded to satellite S_m , otherwise $\alpha_{m,n} = 0$; $\beta_{n,i} = 1$ if sub-channel A_i is allocated for transmitting task ψ_n , otherwise, $\beta_{n,i} = 0$.

A. Transmission Model

Since the distance from a satellite to an IoT terminal is much longer than that between any two IoT terminals, we assume that all the IoT terminals have approximately the same distance to the same satellite. The distance from an IoT terminal to satellite S_m is denoted by d_m and derived as [12]:

$$d_m = \sqrt{R_e^2 + (R_e + R_h)^2 - 2R_e(R_e + R_h)\cos\theta_m}, \quad (1)$$

$$\theta_m = \arccos\left(\frac{R_e}{R_e + R_h} \cdot \cos e_m\right) - e_m, \quad (2)$$

where R_e and R_h represent the radius of the Earth and the height of the satellite orbit above ground, respectively; θ_m is the geocentric angle of satellite S_m , and e_m is the elevation angle of S_m to an IoT terminal with lower limit $e_{\min}$ [13].

The channel gain of sub-channel A_i between satellite S_m and IoT terminal T_n is given by:

$$G_{m,n}^i = G_n \left(\frac{c}{4\pi f_c d_m} \right)^2 (|h_{m,n}^i|)^2, \quad (3)$$

where G_n is the antenna gain (in dBi) of IoT terminal T_n , f_c is the carrier frequency, c denotes the speed of light, and $h_{m,n}^i$ represents the Rician fading with factor K .

If IoT terminal T_n offloads its task to satellite S_m , i.e., $\alpha_{m,n} = 1$, then the data rate from T_n to S_m is given by:

$$C_{m,n} = \sum_{i=1}^I B \log_2 \left(1 + \frac{\alpha_{m,n} \beta_{n,i} P_{n,i} G_{m,n}^i}{\sum_{n' \neq n} \sum_{m'=1}^M \alpha_{m',n'} \beta_{n',i} P_{n',i} G_{m',n'}^i + N_0} \right), \quad (4)$$

where N_0 denotes the additive white Gaussian noise (AWGN) power at the satellite receiver, and $P_{n,i}$ denotes the transmission power from IoT terminal T_n in sub-channel A_i .

In the uplink, the transmission latency $\delta_{m,n}^{\text{Trans}}$ of task ψ_n offloaded to satellite S_m is:

$$\delta_{m,n}^{\text{Trans}} = \frac{\varepsilon_n}{C_{m,n}}. \quad (5)$$

The downlink transmission latency is neglected since the computation result size is typically much smaller than the uplink task data size [6].

B. Problem Formulation

If task ψ_n is processed locally, the computation time is:

$$\delta_n^{\text{IoT}} = \frac{\varepsilon_n \rho}{F_{\text{IoT}}}, \quad (6)$$

If task ψ_n is offloaded to satellite S_m , the computation time is:

$$\delta_{m,n}^{\text{SAT}} = \frac{\varepsilon_n \rho}{F_{m,n}^{\text{SAT}}}, \quad (7)$$

where $F_{m,n}^{\text{SAT}}$ represents the computation resource (in CPU cycle/s) of satellite S_m allocated for processing task ψ_n .

The latency experienced by an IoT terminal includes the transmission latency and computation latency of its task. The latency of terminal T_n is given by:

$$\delta_n = \sum_{m=1}^M \alpha_{m,n} (\delta_{m,n}^{\text{SAT}} + \delta_{m,n}^{\text{Trans}}) + \left(1 - \sum_{m=1}^M \alpha_{m,n} \right) \delta_n^{\text{IoT}}. \quad (8)$$

To minimize the average latency of all IoT terminals, we formulate an optimization problem as follows,

$$\min_{\mathbf{P}, \mathbf{F}, \boldsymbol{\alpha}, \boldsymbol{\beta}} \frac{1}{N} \sum_{n=1}^N \delta_n, \quad (9a)$$

$$\text{s.t. } 0 \leq \sum_{i=1}^I \beta_{n,i} P_{n,i} \leq P_{\text{IoT}}, \forall n \in \{1, \dots, N\}, \quad (9b)$$

$$0 \leq \sum_{n=1}^N \alpha_{m,n} F_{m,n}^{\text{SAT}} \leq F_{\text{SAT}}, \forall m \in \{1, \dots, M\}, \quad (9c)$$

$$\sum_{m=1}^M \alpha_{m,n} \leq 1, \forall n \in \{1, \dots, N\}, \quad (9d)$$

$$\delta_n \leq \Delta, \forall n \in \{1, \dots, N\}, \quad (9e)$$

$$\alpha_{m,n}, \beta_{n,i} \in \{0, 1\}, \quad (9f)$$

where $\mathbf{P} = \{P_{n,i} | n = 1, \dots, N; i = 1, \dots, I\}$, $\mathbf{F} = \{F_{m,n}^{\text{SAT}} | m = 1, \dots, M; n = 1, \dots, N\}$, $\boldsymbol{\alpha} = \{\alpha_{m,n} | m = 1, \dots, M; n = 1, \dots, N\}$, and $\boldsymbol{\beta} = \{\beta_{n,i} | n = 1, \dots, N; i = 1, \dots, I\}$. Constraint (9b) limits the total power allocated by a terminal across all sub-channels; (9c) imposes the computational capacity of each satellite; (9d) ensures that a task can be offloaded to at most one satellite; (9e) ensures that a task must be processed within a scheduling period; and (9f) specifies the binary indicators.

III. ALGORITHM DESIGN

The problem in (9) is non-convex due to the discrete solution space imposed by the binary variables [11]. To address this, it is divided into two sub-problems: the satellite computation resource allocation problem and the joint task offloading, power allocation, and sub-channel allocation problem. We first show that the former is convex and obtain a closed-form solution using the Lagrange multiplier method. Then, a novel LLM-based optimizer is proposed to solve the latter.

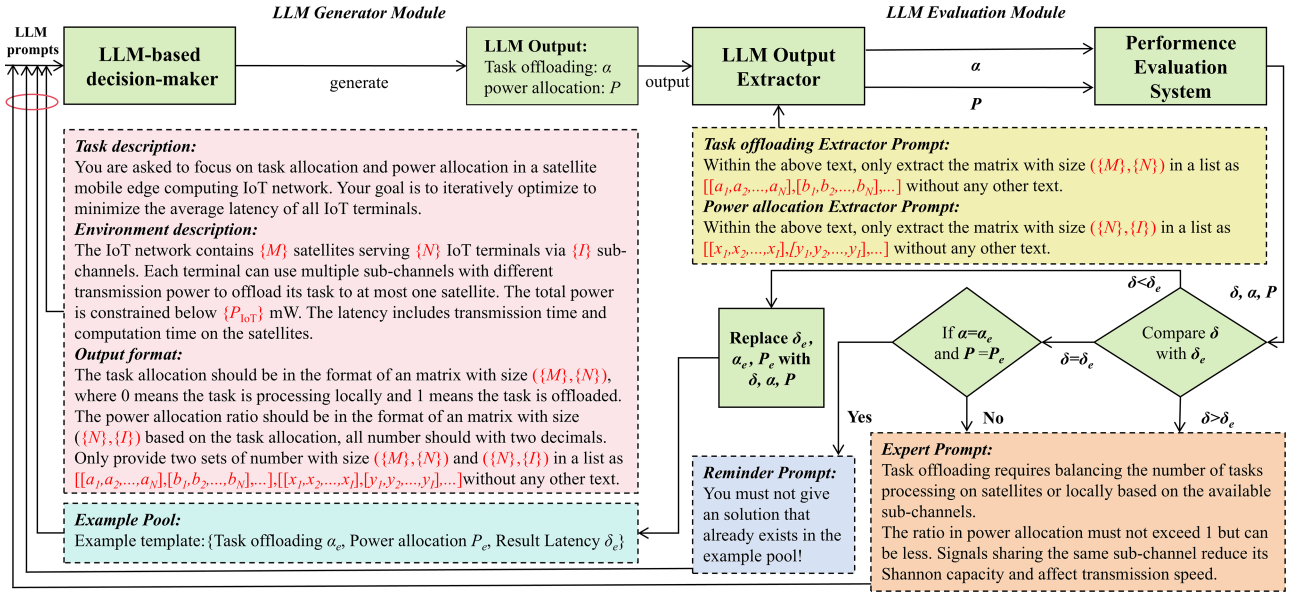


Fig. 2. The LLM-based framework for joint optimization of task offloading, power allocation, and sub-channel allocation.

A. Satellite Computation Resource Allocation

For given values of α , β and P , problem (9) reduces to:

$$\min_{\mathbf{F}} \frac{1}{N} \sum_{n=1}^N \sum_{m=1}^M \alpha_{m,n} \delta_{m,n}^{\text{SAT}}, \quad (10)$$

s.t. (9c).

The second derivative of any element within the summation in (10) with respect to $F_{m,n}^{\text{SAT}}$ is:

$$\frac{\partial^2 (\alpha_{m,n} \delta_{m,n}^{\text{SAT}})}{\partial F_{m,n}^{\text{SAT}2}} = \frac{2\alpha_{m,n} \varepsilon_n \rho}{F_{m,n}^{\text{SAT}3}} \geq 0, \quad (11)$$

where $\alpha_{m,n} \geq 0$, $\varepsilon_n > 0$, $\rho > 0$, and $F_{m,n}^{\text{SAT}} > 0$. So the Hessian matrix of the objective function in (10) is a positive semi-definite matrix, and problem (10) is convex. The Lagrangian function $L(F_{m,n}^{\text{SAT}}, \lambda_m)$ with Lagrange multiplier λ_m is as follows:

$$L(F_{m,n}^{\text{SAT}}, \lambda_m) = \frac{1}{N} \left(\sum_{m=1}^M \sum_{n=1}^N \alpha_{m,n} \delta_{m,n}^{\text{SAT}} \right) + \sum_{m=1}^M \lambda_m \left(\sum_{n=1}^N \alpha_{m,n} F_{m,n}^{\text{SAT}} - F_{\text{SAT}} \right). \quad (12)$$

Taking the partial derivatives of $L(F_{m,n}^{\text{SAT}}, \lambda_m)$ with respect to $F_{m,n}^{\text{SAT}}$ and λ_m , and setting the results to zero, we have:

$$\begin{cases} \frac{\partial L(F_{m,n}^{\text{SAT}}, \lambda_m)}{\partial F_{m,n}^{\text{SAT}}} = -\frac{\alpha_{m,n} \varepsilon_n \rho}{F_{m,n}^{\text{SAT}2}} + \lambda_m \alpha_{m,n} = 0, \\ \frac{\partial L(F_{m,n}^{\text{SAT}}, \lambda_m)}{\partial \lambda_m} = \sum_{n=1}^N \alpha_{m,n} F_{m,n}^{\text{SAT}} - F_{\text{SAT}} = 0. \end{cases} \quad (13)$$

By solving the above equations, we obtain the optimal satellite computation resource allocation:

$$\tilde{F}_{m,n}^{\text{SAT}} = \frac{F_{\text{SAT}} \sqrt{\varepsilon_n \rho}}{\sum_{k=1}^N \alpha_{m,k} \sqrt{\varepsilon_k \rho}}. \quad (14)$$

B. Joint Task Offloading, Power Allocation and Sub-Channel Allocation

For given F , problem (9) reduces to :

$$\min_{\mathbf{P}, \alpha} \frac{1}{N} \sum_{n=1}^N \delta_n, \quad (15a)$$

$$\text{s.t. } 0 \leq \sum_{i=1}^I P_{n,i} \leq P_{\text{IoT}}, \forall n \in \{1, \dots, N\}, \quad (15b)$$

(9c), (9d), (9f),

where for simplicity, the sub-channel allocation indicators $\beta_{n,i}$ are omitted under the assumption that $\beta_{n,i} = 0$ if $P_{n,i} = 0$ and $\beta_{n,i} = 1$ if $P_{n,i} > 0$.

Problem (15) is still non-convex due to the binary constraint and fractional sum terms. To solve it, we propose an LLM-based optimizer as shown in Fig. 2 and detailed below.

The Generator Module consists of an LLM-based decision maker that uses prompts and an example pool as inputs to generate task offloading and power allocation solutions as outputs. The initial prompt includes the task description that outlines the objective based on (15), the environment description that details the system model with key parameters for customization, and the output format that specifies the template for generated solutions. The example pool contains an initial solution, i.e., the best solution to (15) among 100 randomly generated solutions, denoted by α_e , P_e , and δ_e .

The Evaluation Module is composed of an LLM output extractor and a performance evaluation system. To avoid redundant texts due to hallucinations, the LLM extractor uses a task offloading extraction prompt and a power allocation extraction prompt to extract the intended solutions from the output text generated by the LLM. The performance of the extracted solutions α and P is evaluated by substituting them into (15), and calculating the average latency $\delta = (\sum_{n=1}^N \delta_n)/N$; if any constraint of (15) is violated, $\delta = \infty$.

The LLM-based iterative algorithm begins by inputting the initial prompt and example pool into the LLM-based decision maker. The

Algorithm 1: LLM-Based Alternating Optimization.

Input: $M, N, I, \epsilon, \text{max_iterations}$

- 1: Initialize example pool $(\alpha_e, P_e, \delta_e)$ and initial prompt
- 2: Set iteration counter $i \leftarrow 0$, no_update_counter $\leftarrow 0$
- 3: $Prompt_i \leftarrow$ initial prompt
- 4: **repeat**
- 5: Query LLM with $Prompt_i$ and extract $(\alpha_{\text{new}}, P_{\text{new}})$
- 6: **if** any constraint of (15) is violated **then**
- 7: Set $\delta_{\text{new}} \leftarrow \infty$
- 8: **else**
- 9: **for** $m = 1 : M$ and $n = 1 : N$ **do**
- 10: $F_{m,n}^{\text{SAT}} \leftarrow (14)$
- 11: **end for**
- 12: Compute average latency: $\delta_{\text{new}} \leftarrow (8)$ and (9)
- 13: **end if**
- 14: **if** $\delta_{\text{new}} < \delta_e$ **then**
- 15: Update: $(\alpha_e, P_e, \delta_e) \leftarrow (\alpha_{\text{new}}, P_{\text{new}}, \delta_{\text{new}})$
- 16: no_update_counter $\leftarrow 0$
- 17: **else if** $(\alpha_{\text{new}}, P_{\text{new}}, \delta_{\text{new}}) = (\alpha_e, P_e, \delta_e)$ **then**
- 18: $Prompt_{i+1} \leftarrow$ initial prompt::reminder prompt
- 19: no_update_counter $\leftarrow$ no_update_counter + 1
- 20: **else**
- 21: $Prompt_{i+1} \leftarrow$ initial prompt::expert prompt
- 22: no_update_counter $\leftarrow$ no_update_counter + 1
- 23: **end if**
- 24: $i \leftarrow i + 1$
- 25: **until** no_update_counter $\geq \epsilon$ or $i \geq \text{max_iterations}$
- 26: **return** $\alpha_e, P_e, F, \delta_e$

decision maker's output is then evaluated by the Evaluation Module, which compares δ with the average latency δ_e of the solution in the example pool and determines how the inputs to the decision maker should be updated in the next iteration as follows:

- **If $\delta > \delta_e$:** An expert prompt, based on domain-specific knowledge, will be input into the Generator Module.
- **If $\delta < \delta_e$:** The extracted solutions α and P replace α_e and P_e in the example pool, respectively.
- **If $\delta = \delta_e$:** If $\alpha = \alpha_e$ and $P = P_e$, a reminder prompt will be input into the Generator Module to prevent the LLM from being trapped in existing solutions; otherwise, the expert prompt will be input into the Generator Module.

The iteration process terminates when the example pool is not updated for ϵ consecutive iterations or when the number of iterations reaches a preset limit. The example pool returns the final solution to problem (15). Based on the solutions to the two sub-problems, an alternating optimization algorithm is devised to solve (9). The pseudo-code is shown in Algorithm 1.

Since most existing commercial LLM APIs are memoryless, both task offloading and power allocation solutions must be generated within a single conversation (i.e., a series of prompt-response exchanges) with the LLM. The example pool can help maintain continuity across iterations. Some open-source localized models like Llama 3 can mitigate this issue, but currently lag behind in inference performance.

IV. SIMULATION RESULTS

This section presents the performance evaluation of the proposed LLM-based alternating optimization algorithm (LLM), building on (14)

TABLE I
SIMULATION PARAMETERS [6], [12], [13]

Notation	Value	Notation	Value	Notation	Value
N_0	-134dBm	ρ_n	100cycle/bit	P_{IoT}	0.2mW
Δ	1s	F_{IoT}	0.5Gcycle/s	ϵ_{min}	10°
B	10MHz	F_{SAT}	20Gcycle/s	R_e	6371km
G_n	3dBi	f_c	3.49GHz	R_h	550km

and the LLM-based optimizer in Fig. 2. It is well recognized that employing commercial LLM APIs introduces additional network latency due to data exchanges with cloud servers, whereas local deployment of LLMs entails significant hardware and maintenance costs. Therefore, commercial APIs are particularly suitable for edge devices with limited computational resources, offering access to high-performance LLMs at relatively low costs. Conversely, local deployment ensures greater data confidentiality and, by interacting directly with local devices, significantly reduces communication overhead, making it preferable in latency-sensitive and/or safety-critical scenarios. Given the substantial hardware requirements for deploying high-performance LLMs locally, this study utilizes different LLM models through commercial APIs for performance evaluation. Nevertheless, our proposed algorithm is compatible with local deployment-based LLMs too. By testing three widely used models, GPT-4o, LLaMA3.170B, and DeepSeek-R1-0528 under the same prompt design and algorithmic framework, we observe that only GPT-4o successfully converged to an optimized solution, where both the LLaMA3.170B and DeepSeek-R1-0528 failed to complete the algorithm due to hallucinations. The LLaMA3.170B model sometimes returned wrong dimensions of decision matrices, while the DeepSeek-R1-0528 model often generated non-binary values of α within the first 5 iterations, both failing to complete the algorithm. Therefore, GPT-4o is adopted in the following simulations due to its superior reliability and reasoning performance across iterative optimization tasks. The benchmarks for performance comparison include the GA, where the initial population size is set at 400 with a crossover probability of 0.5 and a mutation probability of 0.2; the Deep Deterministic Policy Gradient (DDPG) algorithm with a greedy exploration strategy, where the exploration rate is 0.05, and both the Actor and Critic networks contain three layers with the learning rate of 10^{-5} ; Random Choice (RC), where each task has an equal probability of being processed locally or offloaded to one of the satellites, and the other optimization variables are uniformly distributed within their allowed value ranges; Processing All Locally (PAL), where each IoT terminal processes its task locally and all the variables in problem (9) are set to zero. In the simulation, the task data size ϵ_n is uniformly distributed between 0.3 MB and 0.6 MB, and other parameters are shown in Table I unless otherwise specified.

Fig. 3 shows the convergence performance of the proposed LLM-based algorithm for different values of temperature t , which is a hyperparameter that controls the sharpness of the LLM's output probability distribution, and for different prompt (DP) wording, in comparison with GA and DDPG. Higher temperature values lead to more diverse and stochastic outputs. Our simulations tested temperature values of 0.5, 0.7, 1, 1.3, and 1.5 and found that for $t = 1.3$ and 1.5, the LLM-based algorithm failed to complete due to occasional dimension mismatches in the generated solutions. For $t = 0.5, 0.7$, and 1, the LLM-based algorithms converge within 40 to 200 iterations, significantly faster than both DDPG and GA. The results indicate that $t = 1$ offers the best balance between convergence

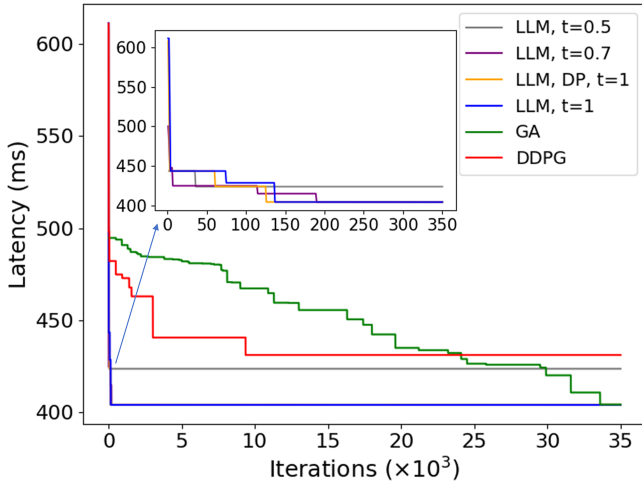


Fig. 3. Convergence of the LLM-based algorithm for different temperature t and different prompt (DP) wording, GA, and DDPG for $M = 3$, $N = 10$, $I = 4$.

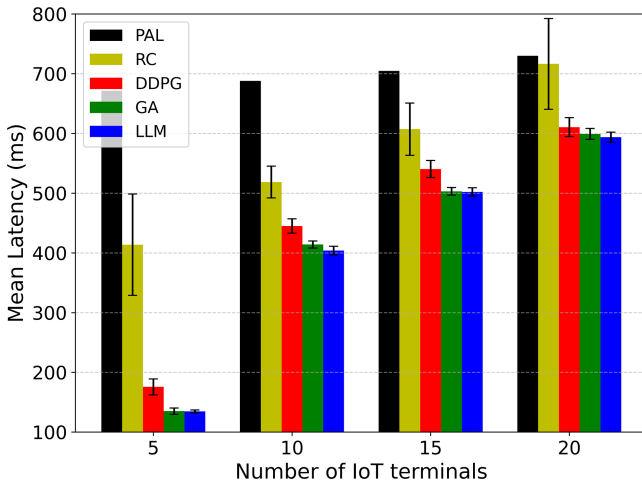


Fig. 4. Mean latency and standard deviation vs. the number of IoT terminals for $M = 3$, $I = 4$.

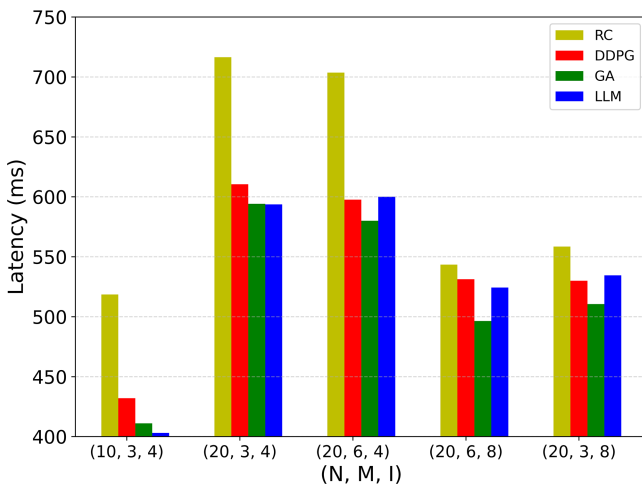


Fig. 5. Average latency for different values of (N, M, I) .

speed and latency minimization performance. Therefore, in the subsequent experiments, the LLM-based algorithm adopts $t = 1$. Additionally, the performance of the LLM-based algorithm with different prompt wording is close to that with the prompt wording defined in Fig. 2, suggesting that the LLM-based algorithm is not sensitive to the wording of prompts as long as the underlying intent remains consistent.

Fig. 4 shows the mean latency and standard deviation achieved by different algorithms across 10 experimental runs for 5, 10, 15, and 20 IoT terminals served by 3 satellites. It shows that as the number of terminals increases, the average latency rises for all the considered schemes. This is mainly due to increased co-channel interference. It also shows that the proposed LLM-based algorithm achieves the lowest mean latency and smallest standard deviation for every considered number of IoT terminals, followed by GA and DDPG. Although DDPG has been widely used for solving non-convex problems, it is prone to being stuck in local optima and is highly sensitive to hyperparameter settings. In contrast, the proposed LLM-based algorithm encourages broader solution exploration when repetitive solutions are detected and avoids getting stuck in local optima through prompt-based adaptations.

Fig. 5 shows the average latency achieved by four schemes for different values of (N, M, I) . The proposed LLM-based algorithm consistently outperforms RC across all scenarios. For the scenarios of $(20, 6, 4)$, $(20, 6, 8)$, and $(20, 3, 8)$, as the decision matrices contain significantly more float numbers, increasing the likelihood of hallucinations or invalid outputs, the LLM-based algorithm is outperformed by GA.

V. CONCLUSIONS AND FUTURE WORK

In this work, we have proposed a novel LLM optimizer combined with the Lagrange multiplier method to minimize the average latency of IoT terminals in a multi-satellite MEC network. Simulation results demonstrate that the LLM-based alternating optimization algorithm converges significantly faster than both GA and DDPG while obtaining a lower average latency for the IoT terminals. The LLM-based algorithm exhibits strong adaptability and effectively achieves the optimization objective across varying numbers of IoT terminals and satellites. Our results also show that for high-dimensional scenarios, it would be necessary to adopt a multi-agent approach—a promising direction for future work. In addition, we plan to investigate other practical issues in satellite MEC networks, such as the response time of LLMs. It is mainly determined by the latency of generating the first token and the subsequent time required to produce the full output text, which scales with the number of output tokens [14].

REFERENCES

- [1] F. Guo, F. R. Yu, H. Zhang, X. Li, H. Ji, and V. C. M. Leung, "Enabling massive IoT toward 6G: A comprehensive survey," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 11891–11915, Aug. 2021.
- [2] W.-C. Chien, C.-F. Lai, M. S. Hossain, and G. Muhammad, "Heterogeneous space and terrestrial integrated networks for IoT: Architecture and challenges," *IEEE Netw.*, vol. 33, no. 1, pp. 15–21, Jan./Feb. 2019.
- [3] M. De Sanctis, E. Cianca, G. Araniti, I. Bisio, and R. Prasad, "Satellite communications supporting Internet of Remote Things," *IEEE Internet Things J.*, vol. 3, no. 1, pp. 113–123, Feb. 2016.
- [4] K. Zhang et al., "Energy-efficient offloading for mobile edge computing in 5G heterogeneous networks," *IEEE Access*, vol. 4, pp. 5896–5907, 2016.
- [5] X. Gao et al., "Hierarchical dynamic resource allocation for computation offloading in LEO satellite networks," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19470–19484, Jun. 2024.

- [6] L. Zhao, Y. Liu, A. Hawbani, N. Lin, W. Zhao, and K. Yu, "QoS-aware multi-hop task offloading in satellite-terrestrial edge networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31453–31466, Oct. 2024.
- [7] S. Zhang, G. Cui, Y. Long, and W. Wang, "Joint computing and communication resource allocation for satellite communication networks with edge computing," *China Commun.*, vol. 18, no. 7, pp. 236–252, Jul. 2021.
- [8] G. Cui, Y. Long, L. Xu, and W. Wang, "Joint offloading and resource allocation for satellite assisted Vehicle-to-Vehicle communication," *IEEE Syst. J.*, vol. 15, no. 3, pp. 3958–3969, Sep. 2021.
- [9] S. Xu, C. K. Thomas, O. Hashash, N. Muralidhar, W. Saad, and N. Ramakrishnan, "Large multi-modal models (LMMs) as universal foundation models for AI-native wireless systems," *IEEE Netw.*, vol. 38, no. 5, pp. 10–20, Sep. 2024.
- [10] L. Qiao, M. B. Mashhadi, Z. Gao, C. H. Foh, P. Xiao, and M. Bennis, "Latency-aware generative semantic communications with pre-trained diffusion models," *IEEE Wireless Commun. Lett.*, vol. 13, no. 10, pp. 2652–2656, Oct. 2024.
- [11] Y. Zhang, H. Zhang, K. Sun, J. Huo, N. Wang, and V. C. M. Leung, "Partial computation offloading in satellite-based three-tier cloud-edge integration networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 2, pp. 836–847, Feb. 2024.
- [12] Z. Zhao, M. Feng, C. Ke, Z. Chen, and T. Jiang, "Federated deep recurrent Q-learning for task partitioning and resource allocation in satellite mobile edge computing assisted industrial IoT," *IEEE Internet Things J.*, vol. 11, no. 15, pp. 26444–26458, Aug. 2024.
- [13] J. M. Gongora-Torres, C. Vargas-Rosales, A. Aragón-Zavala, and R. Villalpando-Hernandez, "Link budget analysis for LEO satellites based on the statistics of the elevation angle," *IEEE Access*, vol. 10, pp. 14518–14528, 2022.
- [14] H. Zhou et al., "Generative AI as a service in 6G edge-cloud: Generation task offloading by in-context learning," *IEEE Wireless Commun. Lett.*, vol. 14, no. 3, pp. 711–715, Mar. 2025.